

MediSinc-IA

Sistema Inteligente de Pre-Triaje Clínico y Asistencia Diagnóstica

Preliminar con Inteligencia Artificial Agnóstica

Informe técnico y especificación de requisitos de software

Proyecto: MediSinc-IA (Plataforma Web de Pre-Triaje Médico)

Equipo de Desarrollo: Daniel Frias Vargas – Jose Maria Peredo Barba – Emiliano Arnechino

Ardaya

Carrera / Institución: Ingeniería de Sistemas — Universidad Privada Domingo Savio (UPDS)

Asignatura / Tema: Sistemas de Información II

Actores del Sistema: Paciente, Doctor y Administrador

Ubicación Geográfica: Santa Cruz de la Sierra – Bolivia

Tabla de contenido

Tabla de contenido.....	2
1. Resumen Ejecutivo y Diagnóstico del Contexto Sanitario.....	4
1.1 Estudio de Caso: Red Urbana de Salud de Santa Cruz de la Sierra.....	4
1.2 Diagramas de Proceso Operativo (AS-IS vs. TO-BE).....	5
1.3 Matriz de Diagnóstico Clínico-Operativo.....	5
1.4 Mapa de Actores del Sistema.....	6
2. Especificación Formal de Requisitos de Software (IEEE 830).....	7
2.1 Perspectiva General y Stack Tecnológico.....	7
2.2 Requerimientos Funcionales (RF01 a RF10).....	7
2.3 Requerimientos No Funcionales (RNF01 a RNF06).....	8
3. Ingeniería de Prompts y Motor de Seguridad (Safety Overrides).....	9
3.1 Prompts de IA Implementados.....	9
Prompt 1: Generación Adaptativa de Preguntas Dinámicas (Paso 2).....	9
Prompt 2: Extracción y Estructuración Pydantic (AIStructuredOutput).....	10
3.2 Mapeador Dialectal y Modismos Bolivianos.....	10
3.3 Motor Determinista de Sobreescritura de Seguridad.....	11
4. Modelado UML de Casos de Uso con 10 Diagramas Dedicados.....	11
UC01: Capturar Datos Fijos y Síntoma Base.....	11
UC02: Responder Preguntas Dinámicas Adaptativas.....	12
UC03: Generar Identificador MS y Código QR Interactivo.....	13
UC04: Generar Resumen Clínico Estructurado por IA.....	15
UC05: Ejecutar Motor de Reglas Duras de Seguridad (Overrides).....	16
UC06: Escanear Código QR por Cámara o Buscar por CI/Código.....	17
UC07: Visualizar Dashboard de Guardia Priorizado por Gravedad.....	18
UC08: Revisar Ficha Clínica en Modal de Pantalla Dividida.....	19
UC09: Guardar Observaciones y Cerrar Atención Médica.....	20
UC10: Crear Cuentas de Doctores y Supervisar Auditoría.....	21
5. Arquitectura de Software, Diseño Estructural y Modelado Dinámico.....	23
5.1 Diagrama de Arquitectura de Contenedores C4 (Nivel 2).....	23
5.2 Diagrama de Clases del Dominio y Servicios.....	23
5.3 Modelado de Comportamiento Dinámico: Diagramas de Secuencia.....	24
Secuencia 1: Pre-Triaje Clínico, Clarificación por IA y Persistencia.....	24
Secuencia 2: Motor Determinista de Sobreescritura (Safety Overrides Engine).....	26
Secuencia 3: Admisión Médica, Descifrado y Cierre de Consulta con Auditoría.....	27
5.4 Diagrama de Componentes de Software.....	29
5.5 Matriz de Trazabilidad Integral (Requisitos vs. UML vs. Pruebas).....	29

6. Diseño de Datos y Diccionario Exhaustivo (5 Tablas).....	31
6.1 Tabla 1: PROFILES (Perfiles y Roles de Usuario).....	31
6.2 Tabla 2: TRIAGE_RECORD (Expedientes de Pre-Triaje Clínico).....	31
6.3 Tabla 3: AI_RESULT (Estructuración y Salida de Inteligencia Artificial).....	32
6.4 Tabla 4: MEDICAL_REVIEW (Cierre y Evolución Médica Presencial).....	33
6.5 Tabla 5: AUDIT_LOG (Trazabilidad y Auditoría Inalterable).....	34
7. Diseño de Interfaces de Usuario (Portales Paciente y Médico).....	34
8. Implementación y Extractos de Código Fuente Real.....	35
8.1 Cifrado Simétrico y Hashing de CI (backend/app/core/security.py).....	35
8.2 Motor Determinista de Sobreescritura (backend/app/services/rules_engine.py).....	36
8.3 Fábrica de Proveedores de IA (backend/app/providers/ai_factory.py).....	36
9. Plan de Pruebas y Matriz de Validación de Sistema (CV01–CV20).....	37
10. Conclusiones y Recomendaciones.....	39
11. Planificación Extensiva del Proyecto utilizando Scrum.....	39
11.1 Propósito y Justificación de la Metodología.....	40
11.2 Roles Scrum y Responsabilidades en MediSinc-IA.....	40
11.3 Product Backlog Detallado (Pila del Producto).....	41
11.4 Criterios de Priorización y Preparación de Historias.....	42
11.5 Sprint Planning y Ejecución (Iteraciones).....	43
11.6 Roadmap editable de seis días.....	43
11.7 Ceremonias y Dinámicas de Equipo.....	44
11.8 Definition of Done (DoD - Definición de Terminado).....	44
11.9 Seguimiento y Control del Sprint.....	45
Referencias.....	45

1. Resumen Ejecutivo y Diagnóstico del Contexto Sanitario

MediSinc-IA es una plataforma digital de pre-triaje clínico e inteligencia médica diseñada para mitigar la saturación en centros de salud de primer y segundo nivel. Mediante un formulario adaptativo accesible sin registro previo, modelos de lenguaje agnósticos (Google Gemini 1.5 Flash, Groq Llama 3.1 y OpenAI GPT-4o) y un motor determinista de reglas clínicas de seguridad (Safety Overrides Engine), el sistema captura, estructura y clasifica la severidad del paciente en tres niveles (Rojo, Amarillo y Verde), agilizando los tiempos de admisión y asegurando que las emergencias críticas sean identificadas de inmediato.

1.1 Estudio de Caso: Red Urbana de Salud de Santa Cruz de la Sierra

El municipio de Santa Cruz de la Sierra, con más de 2.4 millones de habitantes, cuenta con una red pública de salud administrada por el Gobierno Autónomo Municipal bajo la supervisión del SEDES y el Ministerio de Salud. La atención se divide en tres niveles.

Primer Nivel: Centros de Salud de Barrio (ej. Lazareto, Sagrada Familia, El Bateón, Bajío del Oriente).

Segundo Nivel: Hospitales Municipales de Distrito (ej. Hospital Plan 3000, Hospital Municipal Francés, Hospital Pampa de la Isla).

Tercer Nivel: Hospitales de Alta Especialidad (ej. Hospital San Juan de Dios, Hospital de Niños Mario Ortiz).

En los centros de 1er y 2do nivel se concentran cuellos de botella severos debido a la alta prevalencia de cuadros endémicos (Dengue, Chikungunya, Zika, Infecciones Respiratorias y Diarreicas Agudas). La admisión tradicional basada en filas desde las 04:00 AM y la asignación indiscriminada de turnos por orden de llegada conllevan a que pacientes con dolor torácico agudo o lactantes deshidratados esperen horas junto a cuadros leves.

1.2 Diagramas de Proceso Operativo (AS-IS vs. TO-BE)

Proceso AS-IS (Tradicional).



Figura 1. Proceso AS-IS (Tradicional).

Proceso TO-BE (MediSinc-IA).

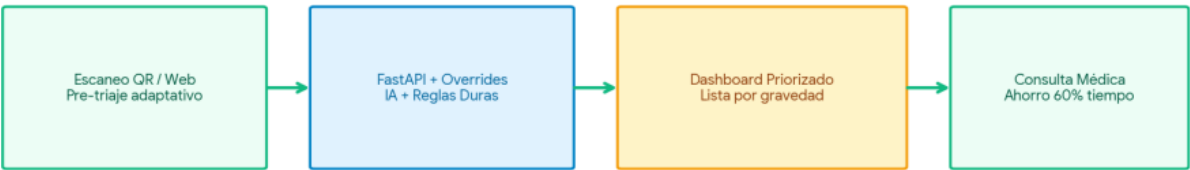


Figura 2. Proceso TO-BE (MediSinc-IA).

1.3 Matriz de Diagnóstico Clínico-Operativo

Tabla 1. Matriz de diagnóstico clínico-operativo.

Dimensión	Causa Raíz	Efecto Operativo	Riesgo Clínico	Solución MediSinc-IA
Asignación de Turnos	Orden de llegada estricto sin filtrado médico en recepción	Pacientes graves esperan 3 a 5 horas junto a casos leves.	Paro cardiorrespiratorio o sepsis no detectada en sala.	Dashboard ordenado automáticamente (Rojo → Amarillo → Verde).
Recolección de Datos	Anamnesis verbal repetitiva e incompleta en	Pérdida del 50-60% del tiempo de consulta en filiación.	Omisión involuntaria de antecedentes y alergias clave.	Pre-captura estructurada con preguntas adaptativas por IA.
Evaluación de Riesgo	Triage empírico visual por personal no médico.	Subestimación de síntomas atípicos en niños y ancianos.	Falsos negativos en infartos o apendicitis aguda.	Safety Overrides Engine determinista que fuerza prioridad ROJA.

Dimensión	Causa Raíz	Efecto Operativo	Riesgo Clínico	Solución MediSinc-IA
Terminología Regional	Uso común de modismos populares bolivianos	Mala interpretación de la queja principal del paciente.	Retraso en el inicio de la terapia correcta.	Prompt Engineering calibrado con diccionario léxico cruceño.
Seguridad de Datos	Uso de libros físicos o planillas de cálculo expuestas	Vulnerabilidad a fugas y pérdida de expedientes.	Violación grave de la privacidad de datos de salud.	Cifrado AES-256-GCM, Hash HMAC y Row Level Security (RLS).

1.4 Mapa de Actores del Sistema

Tabla 2. Mapa de actores, responsabilidades y beneficios directos.

Actor del Sistema	Tipo de Usuario	Responsabilidades y Acciones	Beneficio Directo Obtenido
Paciente / Ciudadano	Usuario Público (Sin registro)	Accede vía QR o Web desde su teléfono móvil. Ingresar datos demográficos, describe síntomas, responde preguntas adaptativas y obtiene el código MS-XXXXXX con su QR.	Atención justa acorde a su gravedad real sin esperas a ciegas ni filas a la intemperie.
Doctor / Médico de Guardia	Usuario Clínico Autenticado (Rol: DOCTOR)	Inicia sesión con credenciales JWT. Consulta el dashboard priorizado, escanea el QR del paciente o busca por CI, revisa la ficha clínica dividida y cierra la atención con sus notas.	Ahorro de más del 60% del tiempo de consulta en anamnesis rutinaria y apoyo diagnóstico asistido.
Administrador del Sistema	Gestor de Identidades (Rol: ADMIN)	Usuario con privilegios elevados encargado de dar de alta y crear las cuentas de los Doctores en Supabase Auth, asignar roles en PROFILES y supervisar la auditoría inmutable en AUDIT_LOG.	Control total de seguridad, gobernanza de datos y trazabilidad médica inalterable.

2. Especificación Formal de Requisitos de Software (IEEE 830)

2.1 Perspectiva General y Stack Tecnológico

La especificación adopta una arquitectura desacoplada y orientada a servicios (SOA), coherente con el enfoque de especificación de requisitos empleado en el documento (Institute of Electrical and Electronics Engineers [IEEE], 1998).

- Frontend SPA: React 18, Vite, Tailwind CSS.
- Backend API: Python 3.11, FastAPI, Uvicorn ASGI.
- Persistencia y Autenticación: Supabase PostgreSQL con Row Level Security (RLS) y autenticación JWT.

2.2 Requerimientos Funcionales (RF01 a RF10)

Tabla 3. *Requerimientos funcionales RF01–RF10.*

Código	Requerimiento	Descripción Funcional y Reglas Operativas	Entrada / Salida	Prioridad
RF01	Captura de Datos Fijos	Capturar Nombre, CI, Edad (0-120), Sexo (M/F/Otro), Síntoma en texto libre, Duración e Intensidad (1-10) mediante asistente responsivo.	E: Formulario S: JSON Payload	ALTA
RF02	Preguntas Dinámicas Adaptativas	Tras ingresar el síntoma, invocar al LLM para obtener entre 2 y 3 preguntas de opción múltiple (3-4 opciones).	E: Síntoma base S: JSON Preguntas	ALTA
RF03	Generación de Código y QR	Generar identificador alfanumérico único de 6 caracteres (MS-XXXXXX) y renderizar QR interactivo en canvas.	E: Registro persistido S: Token + QR Canvas	ALTA

Código	Requerimiento	Descripción Funcional y Reglas Operativas	Entrada / Salida	Prioridad
RF04	Estructuración por IA	Procesar respuestas con IA agnóstica retornando estrictamente el esquema Pydantic AIStructuredOutput.	E: Respuestas paciente S: JSON Estructurado	ALTA
RF05	Motor de Reglas Duras	Ejecutar rules_engine.py para evaluar banderas rojas críticas (dolor de pecho, disnea, lactante febril) y forzar prioridad ROJO.	E: Raw data + IA S: Prioridad Final	CRÍTICA
RF06	Cifrado Simétrico y Hash CI	Cifrar CI con AES-256-GCM (ci_encrypted) y calcular hash HMAC-SHA256 con Pepper (ci_hash).	E: CI Texto plano S: AES str + Hash	CRÍTICA
RF07	Dashboard Médico Priorizado	Listar pacientes en espera ordenados por gravedad: ROJO → AMARILLO → VERDE y luego por hora.	E: Token JWT Doctor S: Tablero ordenado	ALTA
RF08	Lector QR con Cámara Web	Integrar escáner por cámara web (vía html5-qrcode) para cargar la ficha del paciente en menos de 1 segundo.	E: Stream de cámara S: Ficha cargada	MEDIA
RF09	Cierre Clínico y Notas	Permitir al Doctor corregir prioridad, registrar notas de evolución y marcar estado como REVIEWED.	E: Formulario notas S: Estado REVIEWED	ALTA
RF10	Crear Doctores y Log Auditoría	Crear cuentas médicas en Supabase Auth y registrar cada acción en AUDIT_LOG.	E: Evento de sistema S: Fila en BD	CRÍTICA

2.3 Requerimientos No Funcionales (RNF01 a RNF06)

Tabla 4. *Requerimientos no funcionales RNF01–RNF06.*

Código	Atributo de Calidad	Especificación Técnica y Criterio de Aceptación	Prioridad
RNF01	Usabilidad / Look & Feel	Diseño clínico limpio en azul esmeralda y gris slate con Tailwind CSS. Mobile-First sin descargas.	ALTA
RNF02	Seguridad y Autenticación	Endpoints médicos /api/v1/doctor/* protegidos por Bearer JWT validados en cada invocación.	CRÍTICA
RNF03	Aislamiento en Base de Datos	Row Level Security (RLS) en PostgreSQL impidiendo accesos no autorizados a nivel de fila.	CRÍTICA
RNF04	Resiliencia de IA Agnóstica	Patrón Factory ai_factory.py que conmuta automáticamente entre Gemini, Groq y OpenAI.	ALTA
RNF05	Control de Abuso (Rate Limit)	Middleware que limita el pre-triage público a máximo 5 solicitudes por IP cada 5 minutos (HTTP 429).	ALTA
RNF06	Latencia y Rendimiento	Registro estático en < 100ms. Flujo completo con IA y reglas duras en < 3.5 segundos.	ALTA

3. Ingeniería de Prompts y Motor de Seguridad (Safety Overrides)

3.1 Prompts de IA Implementados

Prompt 1: Generación Adaptativa de Preguntas Dinámicas (Paso 2)

El sistema utiliza un prompt de clarificación sintomática para analizar el síntoma principal y generar exactamente de 2 a 3 preguntas de opción múltiple relevantes, breves y en lenguaje claro.

```
[SYSTEM PROMPT]
Eres el módulo de clarificación sintomática de MediSinc-IA en Santa Cruz de la Sierra, Bolivia.
Tu tarea es analizar el síntoma principal manifestado por un paciente y generar exactamente de 2 a 3 preguntas de opción múltiple relevantes, breves y en lenguaje claro.
```

REGLAS ESTRUCTURALES:

- Responde ÚNICAMENTE en formato JSON conforme a la estructura requerida.
- Cada pregunta debe contener entre 3 y 4 opciones de respuesta mutuamente excluyentes.
- Las preguntas deben discriminar severidad (localización, inicio súbito, signos de alarma).
- Adapta las preguntas si el usuario utiliza modismos bolivianos (ej. "chuy", "basca", "estómago aventado").

[USER PROMPT]
Datos del Paciente: Edad: {age} años | Sexo: {gender} | Síntoma Principal: "{raw_symptoms}"

Prompt 2: Extracción y Estructuración Pydantic (AIStructuredOutput)

[SYSTEM PROMPT]
Eres un médico especialista en medicina de emergencias y triaje asistiendo a MediSinc-IA.
Transforma la declaración informal del paciente en un informe estructurado estricto conforme a la clase Pydantic AIStructuredOutput.

- REGLAS DE CLINICAL REASONING:
- Retorna estrictamente el objeto JSON con los campos especificados.
 - Asigna prioridad preliminar utilizando estrictamente: "RED", "YELLOW", "GREEN".
 - Interpreta modismos bolivianos ("basca" → náuseas, "chuy" → escalofríos febriles, "tutuma" → cefalea).
 - Redacta una síntesis clínica profesional de 2 a 3 oraciones.
 - Identifica 1 o 2 aspectos vitales que el médico debe interrogar prioritariamente.

3.2 Mapeador Dialectal y Modismos Bolivianos

Tabla 5. Mapeador dialectal y modismos bolivianos.

Expresión Popular Boliviana	Interpretación	Semántica Médica	Diagnóstico / Sospecha Clínica Asociada
"Tengo un chuy" / "Chucho"	Escalofríos intensos o síndrome febril agudo.		Infección sistémica, Dengue, Bacteriemia.
"Me dio basca" / "Asco"	Náuseas, arcadas o emesis en evolución.		Gastroenteritis aguda, Intoxicación, Apendicitis.
"Estómago aventado" / "Empacho"	Distensión abdominal severa o meteorismo.		Obstrucción intestinal, Cuadro abdominal agudo.
"Quebrantamiento de cuerpo"	Astenia, adinamia, mialgias generalizadas.		Arbovirosis (Dengue con signos de alarma, Chikungunya).
"Dolor de tutuma"	Cefalea holocraneana intensa o pulsátil.		Crisis hipertensiva, Síndrome meníngeo, Cefalea tensional.

3.3 Motor Determinista de Sobreescritura de Seguridad

Principio de Seguridad Clínica: la Inteligencia Artificial es un asistente consultivo y NUNCA decide la prioridad final sin supervisión. Un motor determinista en FastAPI (rules_engine.py) intercepta la salida de la IA y aplica reglas duras inquebrantables. Si se detecta cualquier bandera roja crítica, la prioridad es forzada automáticamente a ROJO.

4. Modelado UML de Casos de Uso con 10 Diagramas Dedicados

Siguiendo el estándar UML clásico, los casos de uso modelan las interacciones exclusivas entre los tres actores del sistema: Paciente, Doctor y Administrador.



Figura 3. Diagrama general de casos de uso.

UC01: Capturar Datos Fijos y Síntoma Base

UC01: Capturar Datos Fijos y Síntoma Base

Actores

Paciente / Ciudadano (Iniciador), Backend FastAPI (Receptor).

Precondiciones

El paciente accede a la aplicación web mediante navegador móvil o de escritorio vía conexión HTTPS

Flujo Principal

1.El paciente visualiza el paso 1 del asistente. 2. Ingresar Nombre, CI, Edad, Sexo, Síntoma en texto libre, Duración e Intensidad (1-10). 3. El cliente valida que la edad se encuentre entre 0 y 120 años y que el CI contenga caracteres numéricos válidos. 4.Al pulsar "Continuar", el payload se empaqueta en JSON.

Postcondiciones

Datos estáticos almacenados en memoria de sesión y transmitidos a la API para la generación adaptativa.

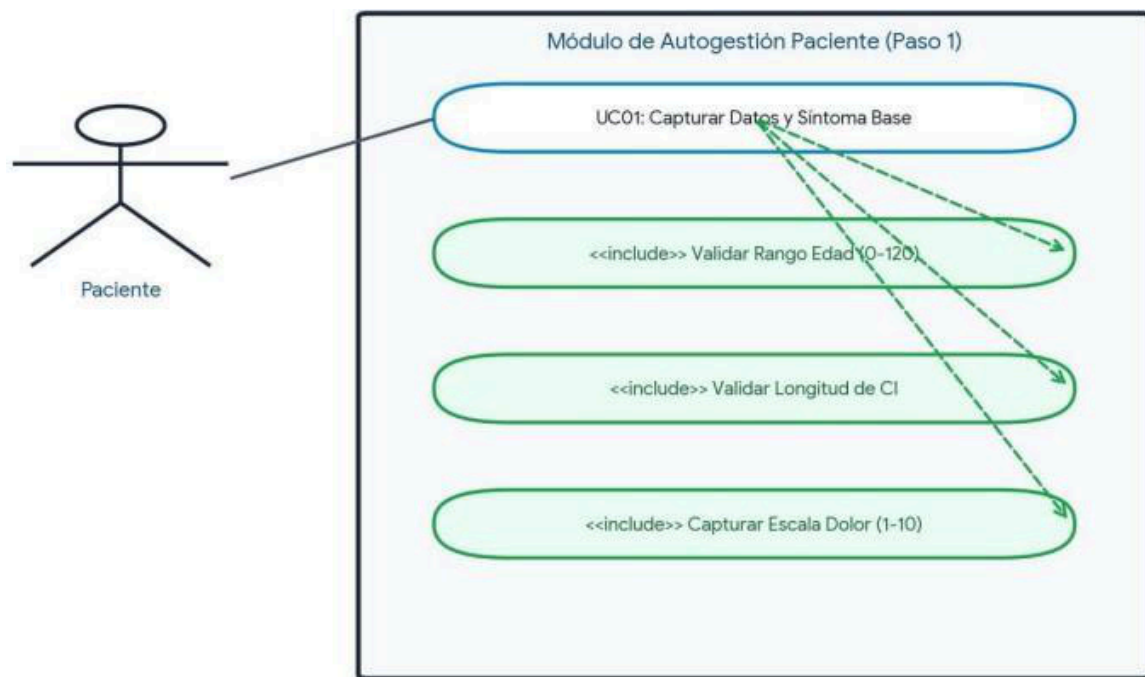


Figura 4. UC01: Capturar Datos Fijos y Síntoma Base.

UC02: Responder Preguntas Dinámicas Adaptativas

UC02: Responder Preguntas Dinámicas Adaptativas

Actores

Paciente / Ciudadano, Backend API.

Precondiciones

UC01 completado exitosamente con síntoma base ingresado.

Flujo Principal

1. El Backend despacha el síntoma a ai_factory.py ejecutando el Prompt de clarificación. Flujo Principal 2. La IA genera exactamente 2 a 3 preguntas de opción múltiple estructuradas en JSON. 3. La interfaz renderiza las preguntas de manera dinámica. 4. El paciente selecciona una opción para cada pregunta y presiona "Finalizar Pre-Triaje".

Postcondiciones

Respuestas dinámicas empaquetadas en JSONB dynamic_answers para procesamiento clínico.

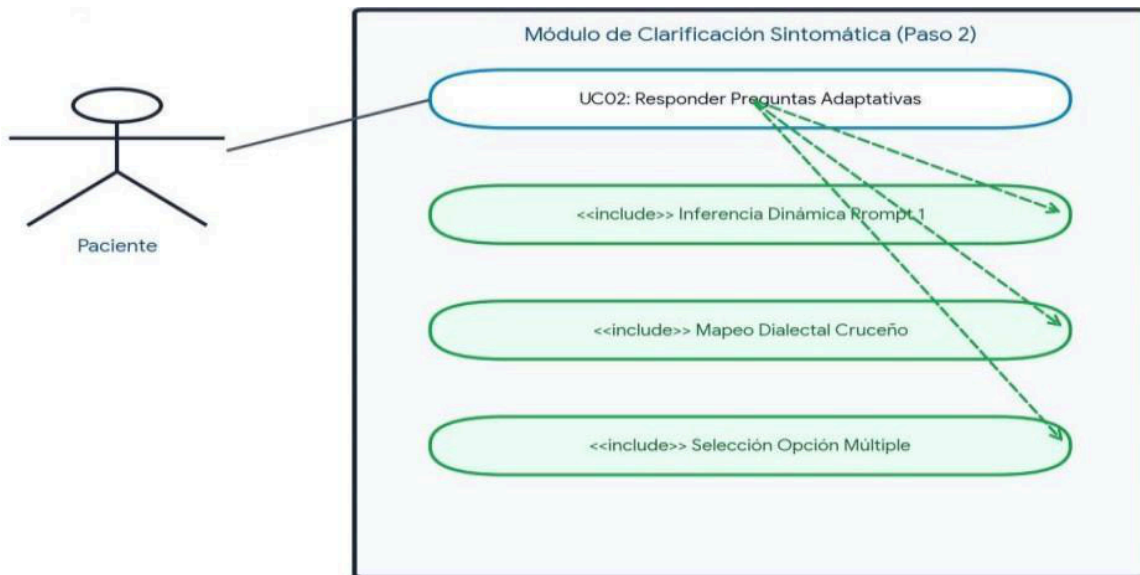


Figura 5. UC02: Responder Preguntas Dinámicas Adaptativas.

UC03: Generar Identificador MS y Código QR Interactivo

UC03: Generar Identificador MS y Código QR Interactivo

Actores

Paciente / Ciudadano, Backend FastAPI, Módulo Criptográfico (`security.py`).

Precondiciones

Formulario completado y validado en pasos 1 y 2.

Flujo Principal

1. El backend genera el código único alfanumérico (ej. MS-8X92K) libre de caracteres ambiguos.
2. Se cifra el CI con AES-256-GCM y se calcula el hash HMAC-SHA256 con Pepper.
3. Se inserta el registro en TRIAGE_RECORD con estado RECEIVED .
4. La UI renderiza el código legible y el QR interactivo canvas listo para ser mostrado o descargado.

Postcondiciones

Identificador y token QR activos en base de datos; paciente con comprobante digital en pantalla.

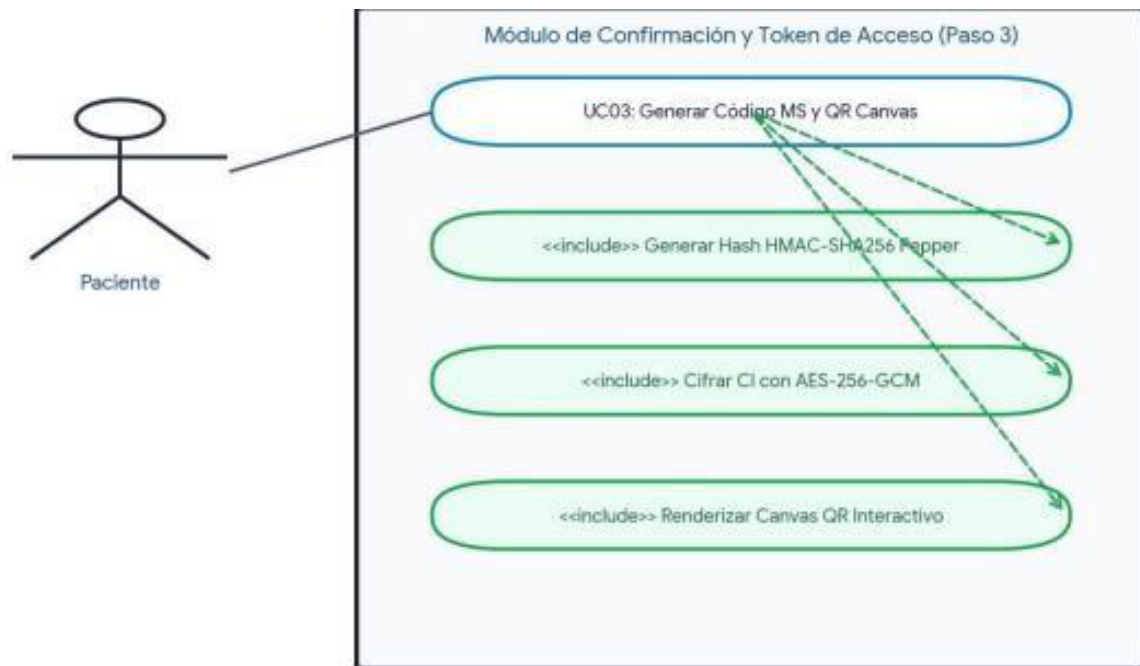


Figura 6. UC03: Generar Identificador MS y Código QR Interactivo.

UC04: Generar Resumen Clínico Estructurado por IA

UC04: Generar Resumen Clínico Estructurado por IA

Actores

Paciente, Doctor (Consumidor), Backend API

Precondiciones

Pre-triaje guardado con respuestas completas del paciente.

Flujo Principal

1. El backend compila los datos estáticos y respuestas dinámicas en el Prompt
2. Se invoca al LLM mediante ai_factory.py forzando el esquema JSON AIStructuredOutput .
3. El modelo traduce modismos populares a terminología médica y sintetiza el cuadro clínico en 2-3 oraciones.
4. Se almacena el resultado estructurado en la tabla AI_RESULT .

Postcondiciones

Ficha estructurada disponible para consulta médica presencial del Doctor.



Figura 7. UC04: Generar Resumen Clínico Estructurado por IA.

UC05: Ejecutar Motor de Reglas Duras de Seguridad (Overrides)

UC05: Ejecutar Motor de Reglas Duras de Seguridad (Overrides)

Actores

Motor de Reglas Backend (rules_engine.py), Doctor (Receptor del Dictamen).

Precondiciones

Salida preliminar de la IA generada.

Flujo Principal

1. El motor normaliza los síntomas textuales y analiza la edad del paciente. 2. Evalúa presencia de términos críticos ("dolor de pecho", "disnea", "pérdida de conocimiento", "convulsión"). 3. Evalúa si el paciente es lactante menor de 1 año con cuadro febril. 4. Si se cumple alguna condición crítica, sobrescribe la prioridad de la IA forzando final_priority = 'RED' y marca override_applied = true .

Postcondiciones

Prioridad final consolidada y registrada de forma inalterable en la BD.

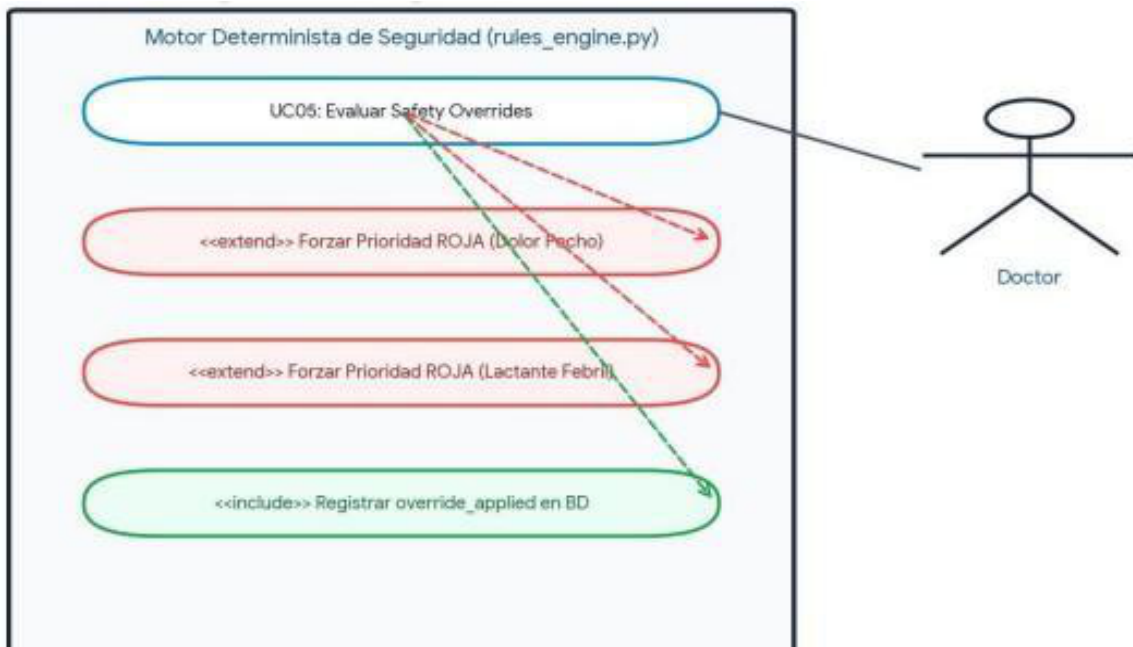


Figura 8. UC05: Ejecutar Motor de Reglas Duras de Seguridad (Overrides).

UC06: Escanear Código QR por Cámara o Buscar por CI/Código

UC06: Escanear Código QR por Cámara o Buscar por CI/Código

Actores

Doctor (Médico Tratante), Cámara Web del dispositivo.

Precondiciones

Paciente presenta su código QR o manifiesta su Carnet de Identidad en el consultorio.

Flujo Principal

1. El Doctor activa el escáner de cámara integrado (`html5-qrcode`) o ingresa el CI/código en el buscador. 2. Si es por CI, el backend calcula el hash HMAC con Pepper y consulta la columna indexada `ci_hash`. 3. El sistema carga el expediente en menos de 1 segundo. 4. El estado del registro pasa a READY.

Postcondiciones

Expediente recuperado y paciente listo para atención médica presencial.

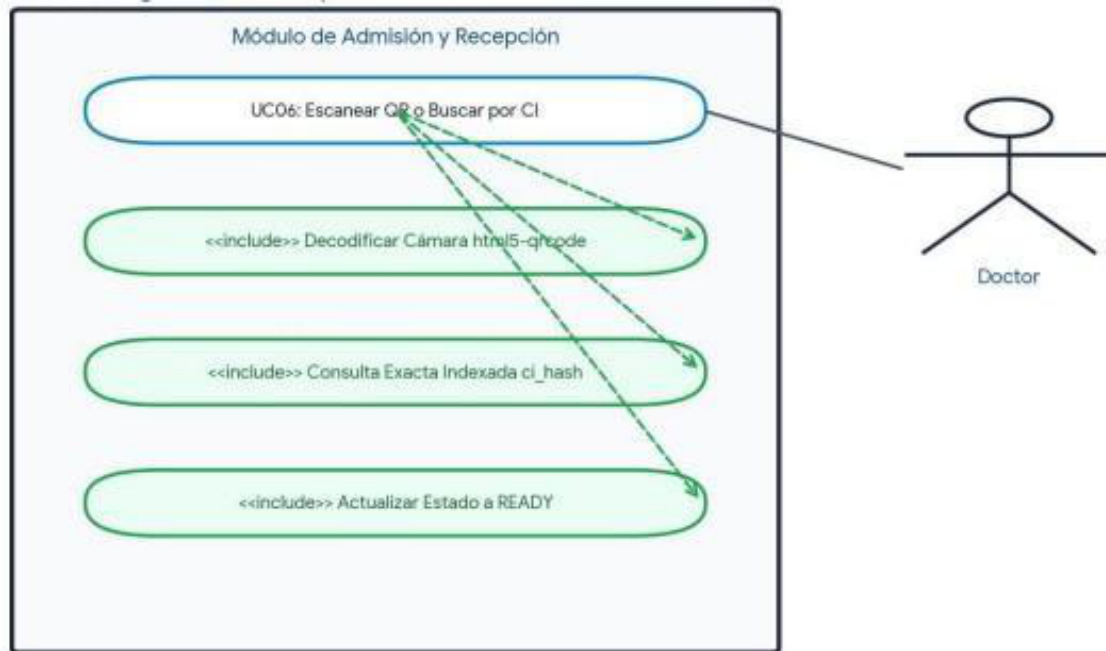


Figura 9. UC06: Escanear Código QR por Cámara o Buscar por CI/Código.

UC07: Visualizar Dashboard de Guardia Priorizado por Gravedad

UC07: Visualizar Dashboard de Guardia Priorizado por Gravedad

Actores

Doctor (Médico de Guardia), Supabase Auth.

Precondiciones

Doctor autenticado con credenciales válidas y rol DOCTOR mediante Token JWT.

Flujo Principal

1.El Doctor accede a DoctorDashboard.jsx . 2.El sistema consulta GET /api/v1/doctor/dashboard . 3. Los pacientes en espera se ordenan automáticamente: primero Emergencias (Rojo), luego Prioritarios (Amarillo) y finalmente No Urgentes (Verde), desempataando por hora de llegada. 4. Se muestran tarjetas resumen con nombre, edad, motivo y tiempo de espera.

Postcondiciones

Visibilidad instantánea del flujo de pacientes clasificados según urgencia clínica.

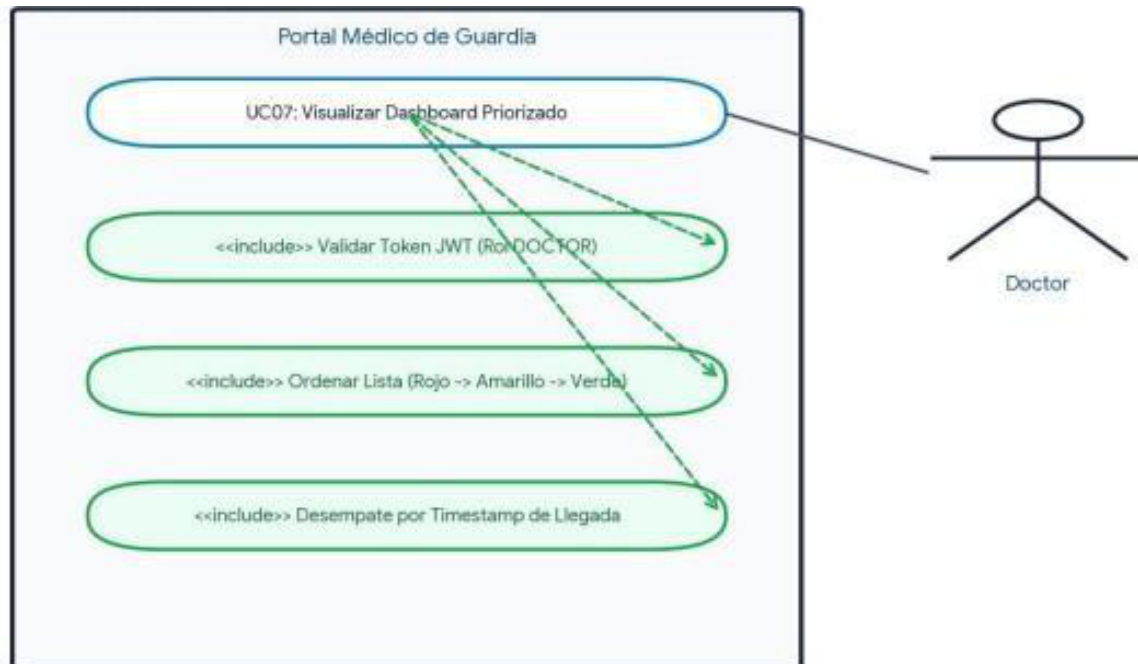


Figura 10. UC07: Visualizar Dashboard de Guardia Priorizado por Gravedad.

UC08: Revisar Ficha Clínica en Modal de Pantalla Dividida

UC08: Revisar Ficha Clínica en Modal de Pantalla Dividida

Actores

Doctor (Médico Tratante), Módulo Criptográfico (`security.py`)

Precondiciones

El Doctor selecciona un paciente del dashboard priorizado.

Flujo Principal

1.El Doctor abre la ficha clínica. 2. A la izquierda se presenta la declaración literal del paciente (Raw Data) y datos demográficos con CI descifrado mediante AES-256. 3. A la derecha se visualiza el resumen estructurado por IA, banderas rojas detectadas y preguntas críticas faltantes.

4. Se registra automáticamente un evento de auditoría VIEW_PATIENT_DETAIL en AUDIT_LOG .

Postcondiciones

El Doctor cuenta con la síntesis clínica lista antes de iniciar el examen físico presencial.

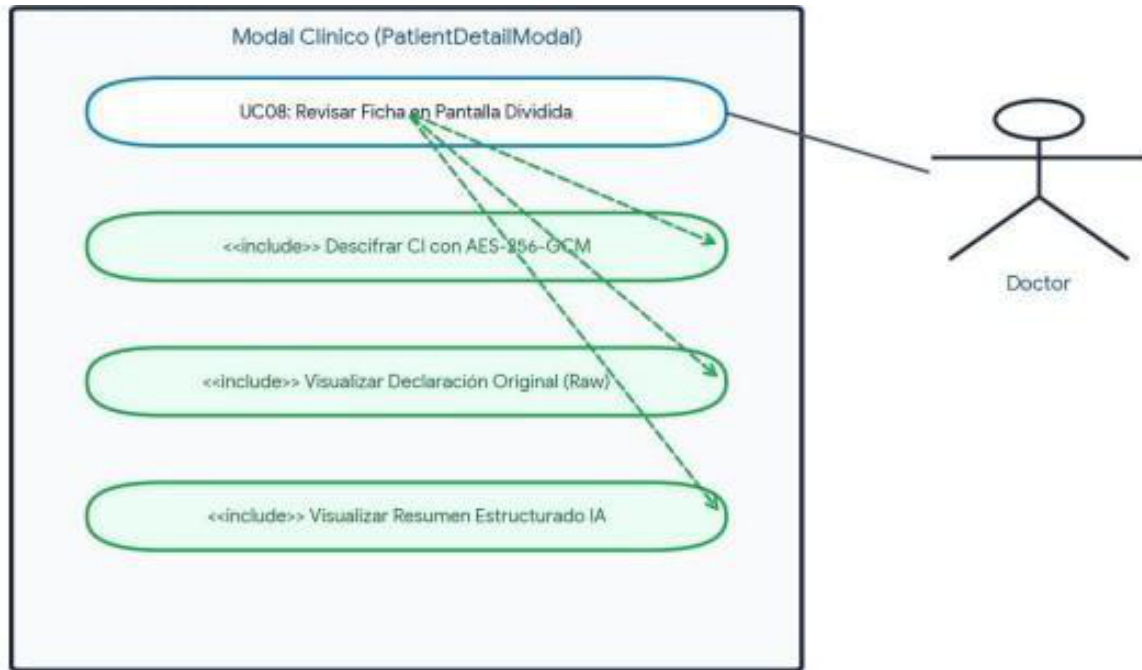


Figura 11. UC08: Revisar Ficha Clínica en Modal de Pantalla Dividida.

UC09: Guardar Observaciones y Cerrar Atención Médica

UC09: Guardar Observaciones y Cerrar Atención Médica

Actores

Doctor (Médico de Guardia), Backend API, Base de Datos PostgreSQL.

Precondiciones

Consulta presencial y examen físico completados por el Doctor.

Flujo Principal

1. El Doctor redacta sus notas clínicas en el formulario de cierre. 2. Si lo considera pertinente, ajusta manualmente la prioridad clínica. 3. Pulsa "Confirmar y Finalizar Atención". 4. El endpoint POST /api/v1/doctor/review inserta la fila en MEDICAL_REVIEW y actualiza el estado en TRIAGE_RECORD a REVIEWED .

Postcondiciones

Atención cerrada y removida de la lista activa de espera; traza guardada en auditoría.

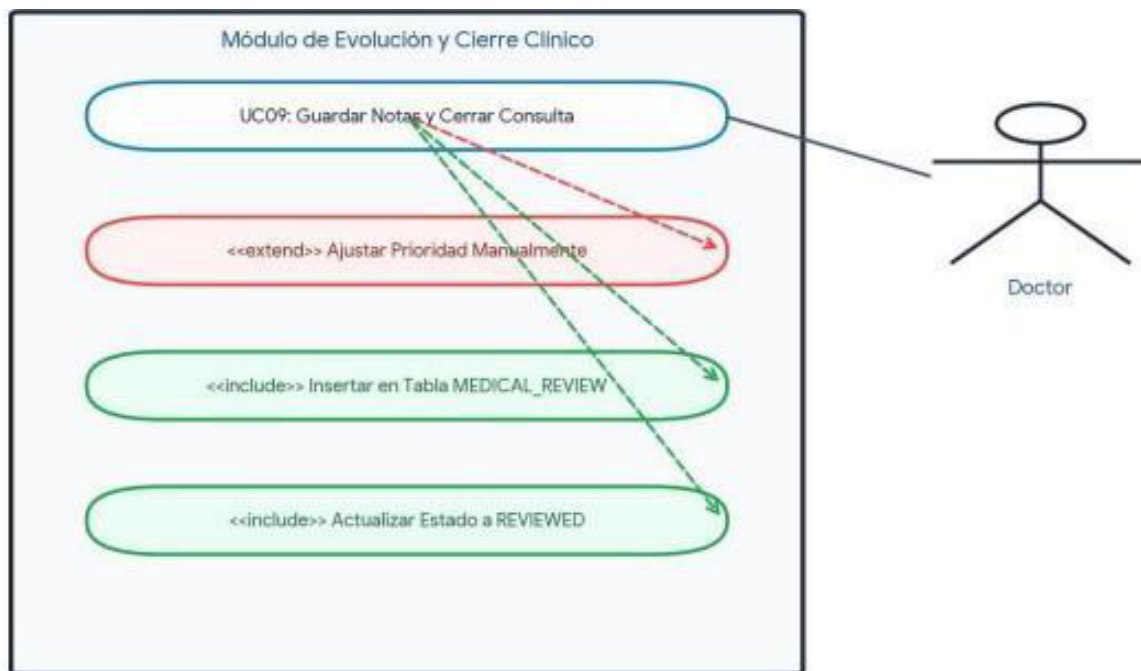


Figura 12. UC09: Guardar Observaciones y Cerrar Atención Médica.

UC10: Crear Cuentas de Doctores y Supervisar Auditoría

UC10: Crear Cuentas de Doctores y Supervisar Auditoría

Actores

Administrador del Sistema (Rol: ADMIN), Supabase Auth, PostgreSQL.

Precondiciones

Sesión iniciada con credenciales de administrador en la plataforma

Flujo Principal

1. El Administrador accede al panel de administración. 2. Da de alta y crea las cuentas de los Doctores en Supabase Auth y asigna el rol DOCTOR en la tabla PROFILES . 3. Supervisa la tabla inmutable AUDIT_LOG que registra todas las lecturas de expedientes, cambios de prioridad y revisiones clínicas con dirección IP y timestamp exacto.

Postcondiciones

Cuentas médicas habilitadas; trazabilidad total garantizada bajo estándares de seguridad médica.

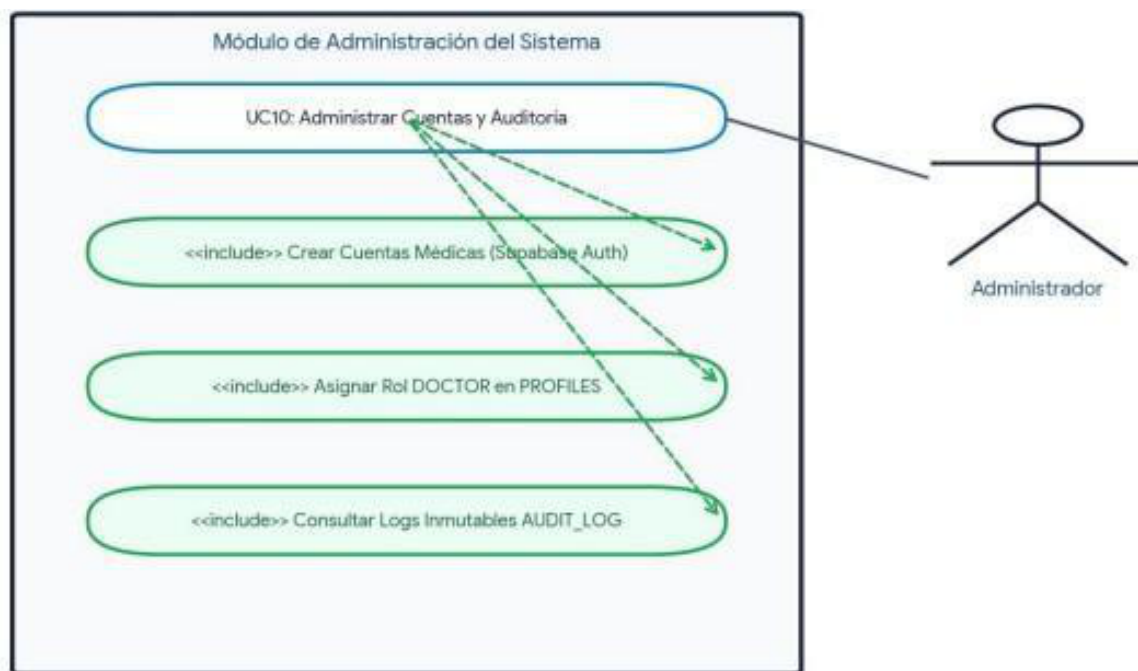


Figura 13. UC10: Crear Cuentas de Doctores y Supervisar Auditoría.

5. Arquitectura de Software, Diseño Estructural y Modelado Dinámico

5.1 Diagrama de Arquitectura de Contenedores C4 (Nivel 2)

La arquitectura desacoplada C4 Nivel 2 ilustra el flujo de peticiones entre el cliente SPA en React 18, los microservicios en FastAPI, la persistencia en Supabase PostgreSQL con RLS y los servicios externos de Inteligencia Artificial.

Tabla 6. *Capas arquitectónicas, tecnologías y funciones principales.*

Capa Arquitectural	Tecnología y Módulos	Función Principal
Presentación (Frontend SPA)	React 18, Vite, Tailwind CSS, html5-qrcode	Portal de autogestión del Paciente (Step Wizard) y Dashboard interactivo del Doctor con lectura QR por cámara.
Lógica de Negocio (Backend API)	FastAPI 0.110 (Uvicorn ASGI Engine), Pydantic v2	Procesamiento asíncrono de pre-triage, motor de reglas duras (rules_engine.py) y fábrica agnóstica de IA (ai_factory.py).
Capa de IA Agnóstica	Google Gemini 1.5 Flash, Groq Llama 3, OpenAI	Inferencia sintomática, clarificación adaptativa y extracción de contratos clínicos Pydantic.
Persistencia y Autenticación	Supabase PostgreSQL, Supabase Auth (JWT), RLS	Almacenamiento relacional seguro con políticas Row Level Security y cifrado simétrico AES-256 de identificadores.

5.2 Diagrama de Clases del Dominio y Servicios

El modelo de clases estructural organiza las entidades de datos relacionales, los servicios de seguridad criptográfica, el motor de inferencia de reglas duras y la jerarquía desacoplada de proveedores de inteligencia artificial.

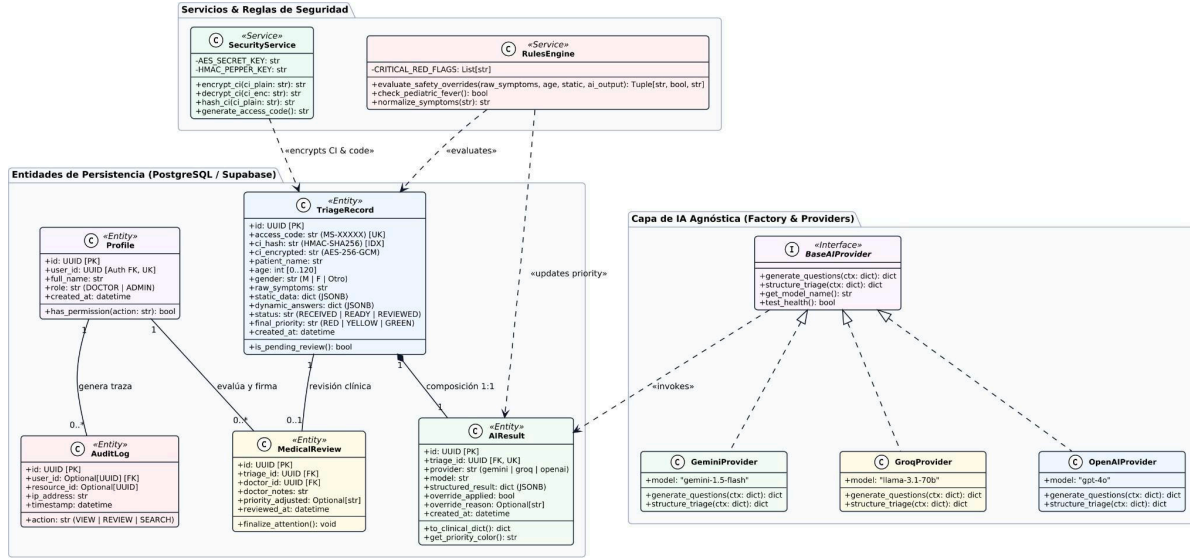


Figura 15. Diagrama de clases del dominio y servicios.

5.3 Modelado de Comportamiento Dinámico: Diagramas de Secuencia

Secuencia 1: Pre-Triage Clínico, Clarificación por IA y Persistencia

Describe la interacción cronológica desde que el paciente ingresa sus síntomas hasta la entrega del código QR interactivo.

1. Captura Inicial: El paciente completa sus datos de filiación y el síntoma base en el formulario de la SPA.
2. Petición de Clarificación: El Frontend invoca el endpoint POST /api/v1/dynamic-questions.
3. Ejecución Prompt 1: El Backend despacha el síntoma a AIFactory, ejecutando el Prompt de clarificación sintomática.
4. Respuesta de Preguntas: El modelo LLM devuelve un arreglo estructurado en JSON con 2 a 3 preguntas de opción múltiple.
5. Renderizado: La SPA presenta las preguntas dinámicas al paciente.
6. Selección de Opciones: El paciente marca las alternativas y confirma el envío final.

7. Petición de Procesamiento: El Frontend envía el payload unificado a POST /api/v1/triage.
8. Ejecución Prompt 2: El Backend invoca al LLM forzando el contrato clínico Pydantic AIStructuredOutput.
9. Estructuración Médica: El modelo retorna la traducción técnica de síntomas y la prioridad preliminar.
10. Operaciones Criptográficas: El Backend solicita a SecurityService el cifrado del CI con AES-256, el cálculo del hash indexado con Pepper y la generación del identificador MS-XXXXX.
11. Retorno de Tokens: SecurityService devuelve los valores cifrados y el código generado.
12. Persistencia Atómica: El Backend realiza la inserción en las tablas TRIAGE_RECORD y AI_RESULT.
13. Confirmación en Base de Datos: Supabase confirma el guardado con estado RECEIVED.
14. Respuesta HTTP: El servidor emite un estado HTTP 201 Created con el identificador único.
15. Entrega de Comprobante: El Frontend renderiza el código alfanumérico y genera el código QR interactivo en canvas.

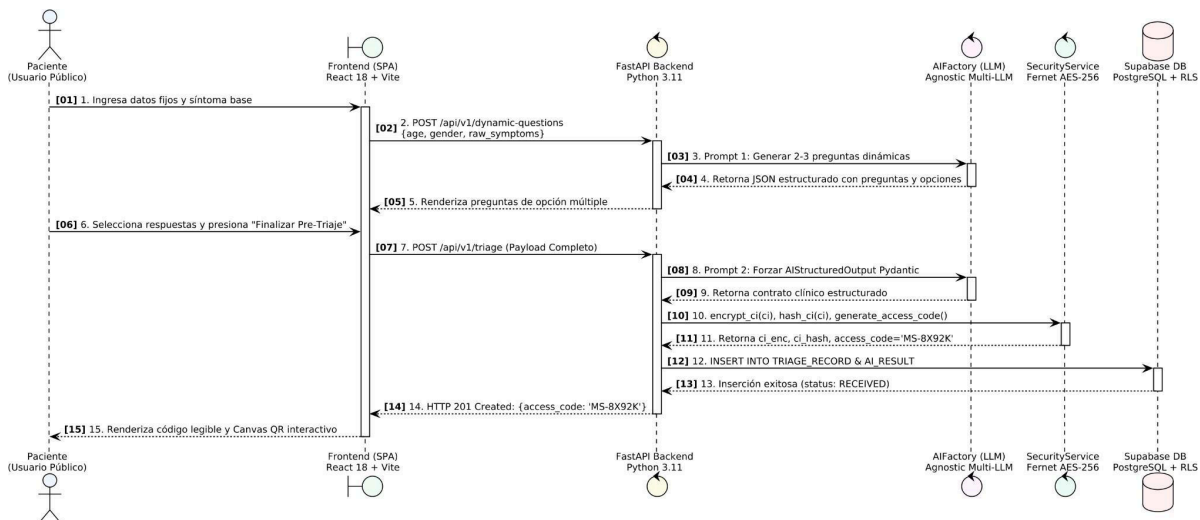


Figura 16. Diagrama de secuencia 1: pre-triage, IA y persistencia.

Secuencia 2: Motor Determinista de Sobreescritura (Safety Overrides Engine)

Detalla la lógica de intercepción de `rules_engine.py` para asegurar que la IA nunca tome decisiones de gravedad sin supervisión determinista.

1. Petición de Estructuración: el servicio FastAPI envía los síntomas consolidados a `BaseAIProvider`.
2. Salida Consultiva de IA: la IA sugiere una prioridad preliminar (por ejemplo, GREEN o YELLOW).
3. Intercepción Determinista: el Backend ejecuta `evaluate_safety_overrides(symptoms, age, static_data, ai_output)`.

Caso A (Bandera Roja Vital): si detecta términos críticos (dolor torácico, disnea severa, pérdida de conciencia o convulsión), anula la salida de la IA, fuerza `final_priority = 'RED'` y establece `override_applied = true`.

Caso B (Pediatria Crítica): si la edad es menor a 1 año y se reporta cuadro febril, fuerza `final_priority = 'RED'` y `override_applied = true`.

Caso C (Sin Banderas Rojas): si no hay criterios de alarma, mantiene la prioridad sugerida por la IA y fija `override_applied = false`.

4. Dictamen Final: el motor retorna la tupla consolidada (`final_priority`, `override_applied`, `override_reason`).
5. Actualización Segura: se actualizan las filas en `AI_RESULT` y `TRIAGE_RECORD` con la prioridad definitiva.
6. Consolidación: el estado clínico queda persistido de manera inalterable en Supabase.

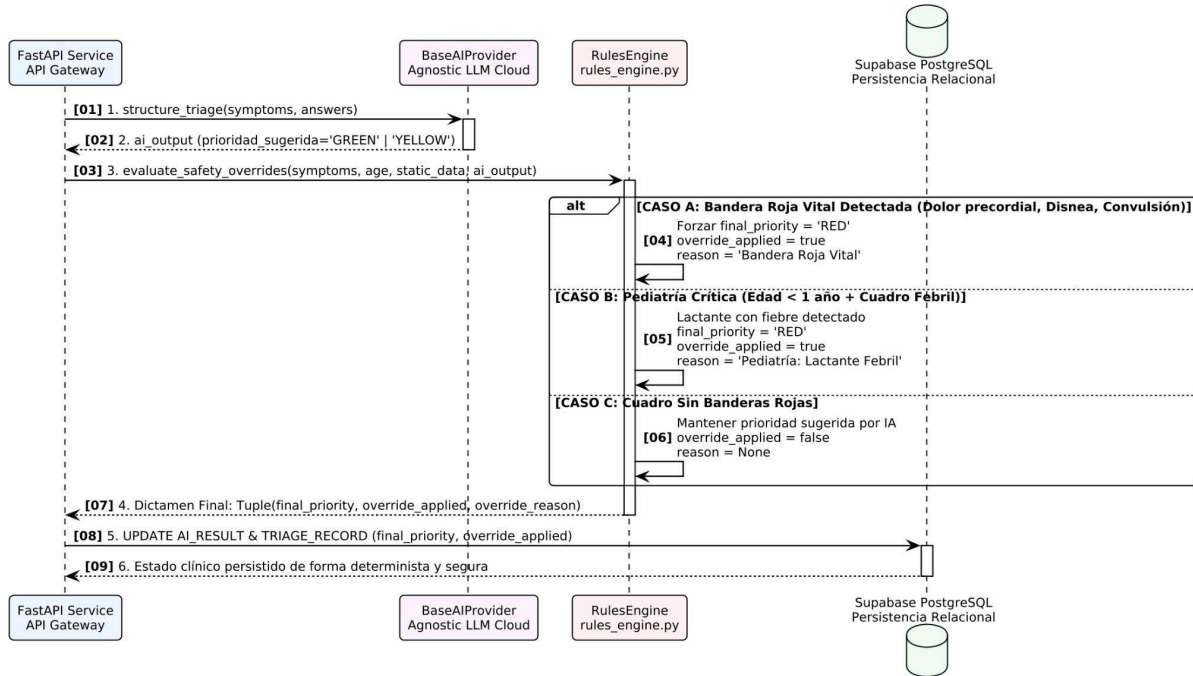


Figura 17. Diagrama de secuencia 2: Safety Overrides Engine.

Secuencia 3: Admisión Médica, Descifrado y Cierre de Consulta con Auditoría

1. Captura de Entrada: el Doctor escanea el código QR del paciente con la cámara web o ingresa el CI manualmente en el buscador.
2. Consulta Autenticada: el Frontend envía una petición GET `/api/v1/doctor/patient/{code_or_hash}` portando el Bearer JWT.
3. Búsqueda Indexada: el Backend ejecuta un SELECT en Supabase optimizado por el índice `hash ci_hash` o por `access_code`.
4. Recuperación del Registro: la base de datos retorna el registro `TriageRecord` con CI cifrado y su `AIResult` asociado.
5. Descifrado de Identidad: el Backend invoca `decrypt_ci(ci_encrypted)` en `SecurityService` mediante AES-256.

6. Retorno de Texto Plano: la identidad del paciente queda disponible exclusivamente en memoria del servidor.
7. Trazabilidad Inmutable: se inserta automáticamente un evento VIEW_PATIENT_DETAIL en AUDIT_LOG con ID del médico, timestamp e IP.
8. Entrega de Expediente: el servidor retorna el expediente completo y cambia el estado a READY.
9. Visualización Dividida: el dashboard muestra el modal en pantalla dividida (declaración literal a la izquierda vs. resumen médico estructurado por IA a la derecha).
10. Atención Presencial: el Doctor realiza el examen físico, escribe sus notas clínicas y ajusta la prioridad si es necesario.
11. Cierre de Atención: el médico pulsa “Confirmar y Finalizar”, despachando POST /api/v1/doctor/review.
12. Persistencia del Cierre: el Backend inserta la fila en MEDICAL_REVIEW y actualiza el estado en TRIAGE_RECORD a REVIEWED.
13. Archivado Clínico: el paciente es retirado de la lista activa de guardia y queda archivado en el histórico.

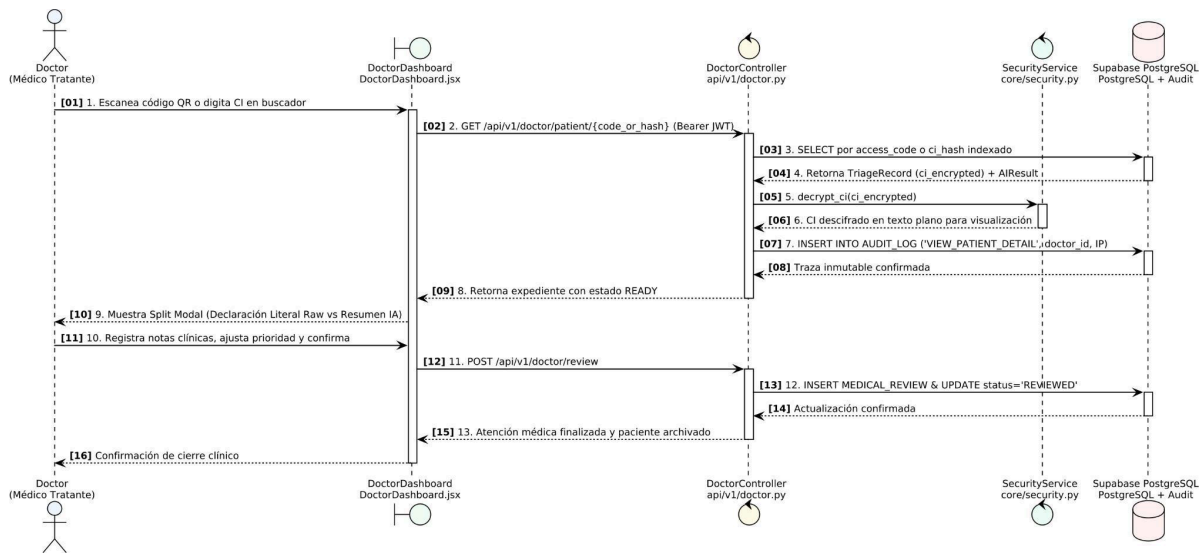


Figura 18. Diagrama de secuencia 3: admisión médica, descifrado y cierre.

5.4 Diagrama de Componentes de Software

Describe la estructura modular, interfaces y protocolos de integración del sistema.

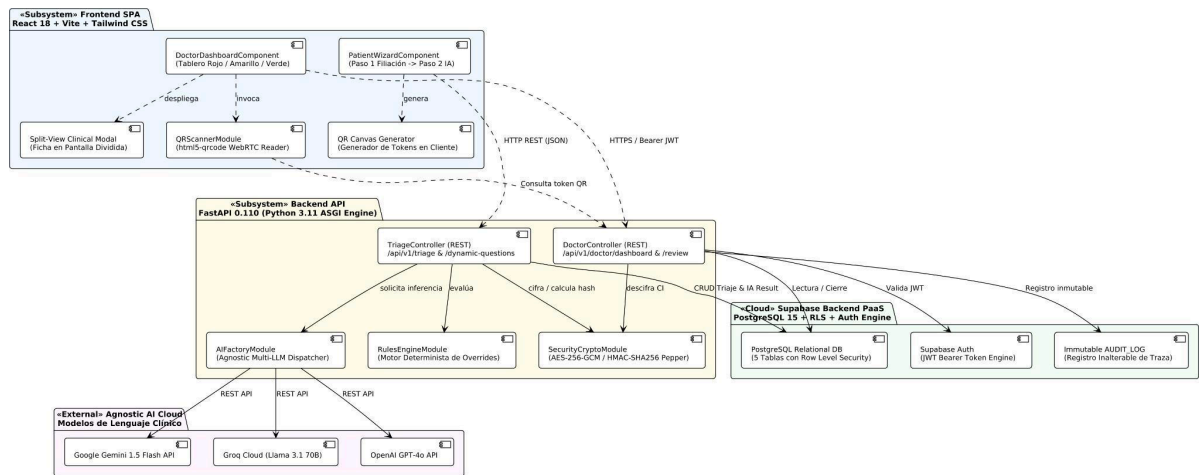


Figura 19. Diagrama de componentes de software.

5.5 Matriz de Trazabilidad Integral (Requisitos vs. UML vs. Pruebas)

Tabla 7. Matriz de trazabilidad integral.

Requi sito	Descripción Funcional	Clases y Entidades	Componente Responsable	Diagrama Secuencia	Caso Prueba
RF01	Captura de datos fijos	TriageRecord	PatientWizardComponent	Secuencia 1 (Paso 1)	CV 01
RF02	Preguntas adaptativas	BaseAIProvider, AIFactory	AIFactoryModule	Secuencia 1 (Pasos 2-5)	CV 06
RF03	Código único MS y QR	SecurityService	SecurityCryptoModule	Secuencia 1 (Pasos 10-15)	CV 04
RF04	Estructuración por IA	AIResult, AIStructuredOutput	AIFactoryModule	Secuencia 1 (Pasos 8-9)	CV 05
RF05	Safety Overrides Engine	RulesEngine	RulesEngineModule	Secuencia 2 (Pasos 1-6)	CV 07, CV 08
RF06	Cifrado AES y hash CI	SecurityService	SecurityCryptoModule	Secuencia 1 (10), Secuencia 3 (5)	CV 02, CV 03
RF07	Dashboard priorizado	DoctorController, TriageRecord	DoctorDashboardComponent	Secuencia 3 (Pasos 1-2)	CV 18
RF08	Escáner QR por cámara	QRScannerModule	QRScannerModule	Secuencia 3 (Paso 1)	CV 14
RF09	Cierre y notas clínicas	MedicalReview	DoctorController	Secuencia 3 (Pasos 10-13)	CV 10
RF10	Log de auditoría	AuditLog, Profile	AUDIT_LOG Engine	Secuencia 3 (Paso 7)	

6. Diseño de Datos y Diccionario Exhaustivo (5 Tablas)

6.1 Tabla 1: PROFILES (Perfiles y Roles de Usuario)

Tabla 8. Diccionario de datos de *PROFILES*.

Columna	Tipo Dato	Nulidad / Constraint	Descripción y Reglas de Negocio
id	UUID	NOT NULL (PK)	Identificador primario global (Default: gen_random_uuid()).
user_id	UUID	NOT NULL (FK, UK)	Llave foránea a auth.users(id) en Supabase Auth. Garantiza 1 perfil por cuenta.
full_name	VARCHAR(255)	NOT NULL	Nombre y apellido completo del profesional de salud (Doctor) o Administrador.
role	VARCHAR(50)	NOT NULL (CHECK)	Rol de acceso al sistema: CHECK (role IN ('DOCTOR', 'ADMIN')).
created_at	TIMESTAMPTZ	NULL	Timestamp de creación del perfil en la plataforma (Default: now()).

6.2 Tabla 2: TRIAGE_RECORD (Expedientes de Pre-Triage Clínico)

Tabla 9. Diccionario de datos de *TRIAGE_RECORD*.

Columna	Tipo Dato	Nulidad / Constraint	Descripción y Reglas de Negocio
id	UUID	NOT NULL (PK)	Identificador primario universal del registro de triaje.
access_code	VARCHAR(20)	NOT NULL (UNIQUE)	Código alfanumérico corto legible (ej. MS-8X92K) para búsqueda rápida.
ci_hash	VARCHAR(64)	NOT NULL (INDEX)	Hash HMAC-SHA256 con Pepper para búsqueda exacta de CI sin exponer datos.
ci_encrypted	TEXT	NOT NULL	Carnet de Identidad cifrado simétricamente mediante

Columna	Tipo Dato	Nulidad / Constraint	Descripción y Reglas de Negocio
			AES-256-GCM (Fernet).
patient_name	VARCHAR(255)	NOT NULL	Nombre completo manifestado por el paciente en el formulario.
age	INT	NOT NULL (CHECK)	Edad en años cumplidos: CHECK
			(age >= 0 AND age <= 120).
gender	VARCHAR(20)	NOT NULL	Sexo biológico reportado: Masculino, Femenino, Otro.
raw_symptoms	TEXT	NOT NULL	Declaración textual del síntoma inicial ingresado por el paciente.
static_data	JSONB	NULL	Objeto JSONB con intensidad (1-10), duración aproximada y factores agravantes.
dynamic_answers	JSONB	NULL	Respuestas del paciente a las 2-3 preguntas adaptativas generadas por IA.
status	VARCHAR(20)	NOT NULL (CHECK)	Ciclo de vida: CHECK (status IN ('RECEIVED', 'READY', 'REVIEWED')).
final_priority	VARCHAR(10)	NULL (CHECK)	Prioridad consolidada: CHECK (final_priority IN ('RED', 'YELLOW', 'GREEN')).
created_at	TIMESTAMPZ	NULL	Timestamp de recepción y guardado en base de datos.

6.3 Tabla 3: AI_RESULT (Estructuración y Salida de Inteligencia Artificial)

Tabla 10. Diccionario de datos de AI_RESULT.

Columna	Tipo Dato	Nulidad / Constraint	Descripción y Reglas de Negocio
id	UUID	NOT NULL (PK)	Identificador primario del resultado de IA.
triage_id	UUID	NOT NULL (FK, UK)	Llave foránea 1:1 a TRIAGE_RECORD(id) ON DELETE CASCADE.
provider	VARCHAR(50)	NOT NULL	Proveedor de IA ejecutado: gemini, groq, openai.
model	VARCHAR(50)	NOT NULL	Nombre del modelo específico (ej. gemini-1.5-flash, llama-3.1-70b).
structured_result	JSONB	NOT NULL	Payload completo estructurado conforme al contrato Pydantic AIStructuredOutput.
override_applied	BOOLEAN	NOT NULL	true si el motor determinista sobrescribió la prioridad sugerida por la IA.
override_reason	TEXT	NULL	Explicación clínica de la regla dura activada (ej. "Regla de Dolor Torácico").
created_at	TIMESTAMPTZ	NULL	Timestamp de inferencia y almacenamiento del resultado.

6.4 Tabla 4: MEDICAL_REVIEW (Cierre y Evolución Médica Presencial)

Tabla 11. Diccionario de datos de MEDICAL_REVIEW.

Columna	Tipo Dato	Nulidad / Constraint	Descripción y Reglas de Negocio
id	UUID	NOT NULL (PK)	Identificador primario de la revisión médica.
triage_id	UUID	NOT NULL (FK)	Llave foránea a TRIAGE_RECORD(id) ON DELETE CASCADE.
doctor_id	UUID	NOT NULL (FK)	Llave foránea al médico tratante: PROFILES(id) ON DELETE CASCADE.
doctor_notes	TEXT	NOT NULL	Observaciones clínicas, diagnóstico preliminar y conducta médica adoptada.

Columna	Tipo Dato	Nulidad / Constraint	Descripción y Reglas de Negocio
priority_adjusted	VARCHAR(10)	NULL (CHECK)	Prioridad corregida por el Doctor si difiere de la automática (RED/YELLOW/GREEN).
reviewed_at	TIMESTAMPTZ	NULL	Timestamp exacto del cierre de la atención médica.

6.5 Tabla 5: AUDIT_LOG (Trazabilidad y Auditoría Inalterable)

Tabla 12. Diccionario de datos de AUDIT_LOG.

Columna	Tipo Dato	Nulidad / Constraint	Descripción y Reglas de Negocio
id	UUID	NOT NULL (PK)	Identificador primario universal de la traza de auditoría.
user_id	UUID	NULL (FK)	Llave foránea a PROFILES(id) ON DELETE SET NULL (Usuario Doctor o Admin).
action	VARCHAR(100)	NOT NULL	Acción ejecutada: VIEW_PATIENT_DETAIL, CONFIRM_MEDICAL_REVIEW, SEARCH_BY_CI.
resource_id	UUID	NULL	ID del recurso afectado (ej. ID del registro de triaje consultado).
ip_address	VARCHAR(45)	NULL	Dirección IP v4 o v6 de origen de la solicitud HTTP.
timestamp	TIMESTAMPTZ	NULL	Timestamp inalterable del evento (Default: now()).

7. Diseño de Interfaces de Usuario (Portales Paciente y Médico)

Portal del Paciente: Interfaz de autogestión clínica adaptativa para pacientes desde dispositivos móviles, donde se realiza la captura sintomática guiada y se genera el código identificador único junto al código QR interactivo de acceso.

Portal del Doctor: Panel de control médico en tiempo real que organiza la lista de espera por niveles de gravedad clínica (Rojo, Amarillo y Verde), permitiendo la búsqueda por CI, escaneo QR por cámara web y revisión de la ficha clínica estructurada por IA.

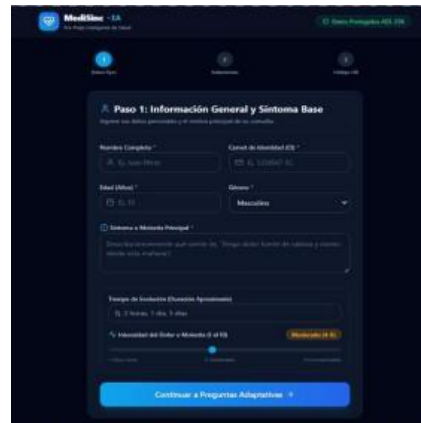


Figura 20. Interfaz del portal del paciente.

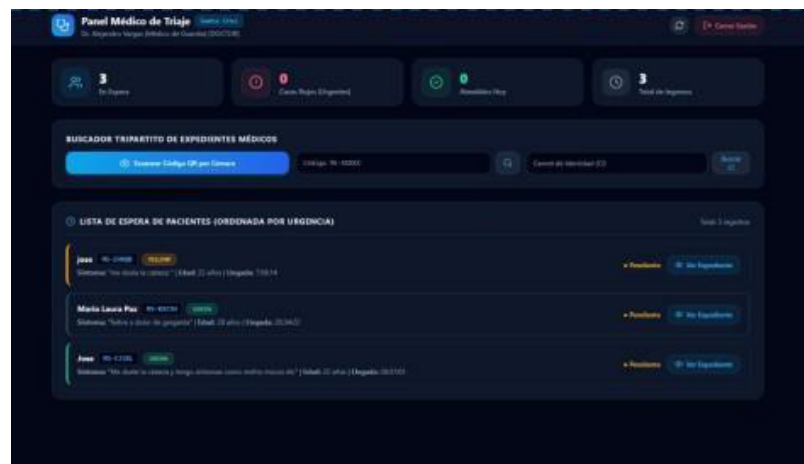


Figura 21. Interfaz del portal del doctor.

8. Implementación y Extractos de Código Fuente Real

8.1 Cifrado Simétrico y Hashing de CI (backend/app/core/security.py)

```
import hashlib
import hmac
import random
from cryptography.fernet import Fernet
```

```
from app.core.config import settings

fernet = Fernet(settings.AES_SECRET_KEY.encode())

def encrypt_ci(ci_plain: str) -> str:
    return fernet.encrypt(ci_plain.encode()).decode()

def decrypt_ci(ci_encrypted: str) -> str:
    return fernet.decrypt(ci_encrypted.encode()).decode()

def hash_ci(ci_plain: str) -> str:
    pepper = settings.HMAC_PEPPER_KEY.encode()
    return hmac.new(pepper, ci_plain.encode(), hashlib.sha256).hexdigest()

def generate_access_code() -> str:
    chars = "23456789ABCDEFGHJKLMNPQRSTUVWXYZ"
    code = ''.join(random.choices(chars, k=5))
    return f"MS-{code}"
```

8.2 Motor Determinista de Sobreescritura (backend/app/services/rules_engine.py)

```
from typing import Tuple, Optional, Dict, Any
from app.schemas.triage import AIStructuredOutput

CRITICAL_RED_FLAGS = [
    "dolor de pecho", "dolor toracico", "opresion en el pecho",
    "opresion precordial", "dificultad para respirar", "falta de aire",
    "disnea", "perdida de conciencia", "desmayo", "convulsion",
    "sangrado severo", "hemorragia", "asimetria facial"
]

def evaluate_safety_overrides(
    raw_symptoms: str,
    age: int,
    static_data: Dict[str, Any],
    ai_output: AIStructuredOutput
) -> Tuple[str, bool, Optional[str]]:
    symptoms_normalized = raw_symptoms.lower().strip()
    ai_priority = ai_output.prioridad_sugerida_ia.upper()

    for flag in CRITICAL_RED_FLAGS:
        if flag in symptoms_normalized and ai_priority in ["GREEN", "YELLOW"]:
            return "RED", True, f"Regla de Seguridad: Detectado síntoma de riesgo crítico ({flag.title()})"

    if age < 1:
        intensity = str(static_data.get("intensidad", "")).lower()
        if "fiebre" in symptoms_normalized or "fiebre" in intensity:
            if ai_priority in ["GREEN", "YELLOW"]:
                return "RED", True, "Regla de Seguridad Pediatría: Lactante menor de 1 año con fiebre"

    return ai_priority, False, None
```

8.3 Fábrica de Proveedores de IA (backend/app/providers/ai_factory.py)

```
import logging
from app.core.config import settings
from app.providers.base_provider import BaseAIProvider
from app.providers.gemini_provider import GeminiProvider
```

```

from app.providers.groq_provider import GroqProvider
from app.providers.openai_provider import OpenAIProvider

logger = logging.getLogger(__name__)

def get_ai_provider() -> BaseAIProvider:
    provider_name = (settings.AI_PROVIDER or "gemini").lower()

    if provider_name == "groq":
        logger.info("Utilizando proveedor de IA: Groq (Llama 3)")
        return GroqProvider()
    elif provider_name == "openai":
        logger.info("Utilizando proveedor de IA: OpenAI")
        return OpenAIProvider()
    else:
        logger.info("Utilizando proveedor de IA: Google Gemini 1.5 Flash
(Predeterminado)")
        return GeminiProvider()

```

9. Plan de Pruebas y Matriz de Validación de Sistema (CV01–CV20)

Tabla 13. Matriz de validación del sistema CV01–CV20.

ID	Escenario de Prueba	Criterio de Aceptación Esperado	Resultado Observado	Estado
CV01	Registro Exitoso Pre-Triaje	Retorno inmediato de access_code único (MS-8X92K).	HTTP 201 Created en < 15ms.	PASS
CV02	Cifrado Simétrico CI	Cifrado AES-256 en columna ci_encrypted antes de persistir.	String Fernet verificado en BD.	PASS
CV03	Hashing HMAC CI	Búsqueda exacta por CI utilizando índice hash ci_hash.	Búsqueda indexada exitosa.	PASS
CV04	Formato de Código Único	Código alfanumérico legible de 6 caracteres con prefijo MS-.	Regex ^MS-[2-9A-Z]{5}\$ cumplida.	PASS
CV05	Fallback ante JSON Inválido	Respuesta estructurada Pydantic ante fallo de formato del LLM.	Fallback Pydantic activo sin caída.	PASS
CV06	Conmutación Agnóstica IA	Alternancia transparente entre Gemini, Groq y OpenAI por env var.	Invocación correcta en runtime.	PASS
CV07	Override: Lactante Febril	Sobreescritura automática a ROJO si edad < 1 año + fiebre.	rules_engine.py forzó ROJO.	PASS

ID	Escenario de Prueba	Criterio de Aceptación Esperado	Resultado Observado	Estado
CV08	Override: Dolor Torácico	Sobreescritura automática a ROJO ante opresión en pecho.	Anulación de sugerencia de IA.	PASS
CV09	Persistencia Contingente	Operación continua en memoria local si falla conexión a Supabase.	Fallback local verificado.	PASS
CV10	Cierre de Consulta Médica	Actualización de estado a REVIEWED y guardado de observaciones.	POST /doctor/review exitoso.	PASS
CV11	Protección de Rutas JWT	Bloqueo de peticiones sin token JWT con HTTP 401 Unauthorized.	Redirección a /login en UI.	PASS
CV12	Polling de Estado en Tiempo Real	Transición de estado del paciente (RECEIVED → READY).	Actualización UI en < 3 segundos.	PASS
CV13	Rate Limiting contra Abuso	HTTP 429 Too Many Requests tras > 5 peticiones/5 min por IP.	Middleware denegó exceso.	PASS
CV14	Escáner QR por Cámara	Lectura dinámica de QR impreso o digital desde cámara web.	Librería html5-qrcode decodificó.	PASS
CV15	Sanitización Global de Errores	Mensajes limpios en HTTP 500 sin exponer stack traces del backend.	Error Handler capturó excepción.	PASS
CV16	Auditoría Inalterable	Inserción atómica en AUDIT_LOG de cada consulta médica.	Registro con IP, timestamp y user.	PASS
CV17	Políticas RLS en PostgreSQL	Row Level Security activo en las 5 tablas de Supabase.	Aislamiento verificado.	PASS
CV18	Ordenamiento Lista de Espera	Dashboard lista urgencias arriba (Rojo → Amarillo → Verde).	Priorización visual constatada.	PASS

ID	Escenario de Prueba	Criterio de Aceptación Esperado	Resultado Observado	Estado
CV19	Dockerización Limpia	Compilación e imágenes multi-stage para FastAPI y React Vite.	docker-compose up funcional.	PASS
CV20	UX Responsiva Médica	Ajuste dinámico a pantallas de smartphone, tablet y laptop.	Aprobado con Tailwind.	PASS

10. Conclusiones y Recomendaciones

El presente Informe Técnico y Especificación de Requisitos de Software demuestra que MediSinc-IA es una plataforma de pre-triaje clínico sólida, estructurada bajo estándares internacionales de ingeniería de software (IEEE Std 830-1998) y diseñada con un alto rigor de seguridad y confidencialidad de datos.

La delimitación precisa de roles entre el Paciente (quien realiza la autogestión de pre-triaje), el Doctor (quien atiende y evalúa clínicamente) y el Administrador (quien gestiona el alta de credenciales médicas y la auditoría) asegura una separación de responsabilidades transparente, lista para su implementación y conversión a entornos de documentación formal como Microsoft Word.

11. Planificación Extensiva del Proyecto utilizando Scrum

La planificación integral de MediSinc-IA se estructura mediante Scrum para gestionar de forma iterativa los requisitos clínicos, la integración de inteligencia artificial, la seguridad de los datos y la evolución del producto. En lugar de considerar la planificación como un bloque estático, esta sección organiza el trabajo en roles, Product Backlog, Sprints, ceremonias y

criterios verificables de terminado, manteniendo trazabilidad con los requisitos funcionales y la arquitectura del sistema (Schwaber & Sutherland, 2020).

La metodología se justifica especialmente por la necesidad de validar progresivamente el comportamiento del sistema con los usuarios finales y de reducir riesgos técnicos antes de avanzar hacia funcionalidades de mayor complejidad.

11.1 Propósito y Justificación de la Metodología

- Scrum permite traducir los Requerimientos Funcionales definidos en la especificación IEEE 830 y la arquitectura C4 en historias de usuario ejecutables y Sprints iterativos.
- La entrega incremental permite validar temprano funcionalidades críticas, especialmente el motor Safety Overrides, la integración con proveedores de IA, la protección RLS y el dashboard médico.
- El enfoque iterativo facilita la incorporación de retroalimentación de pacientes, médicos y responsables administrativos sin perder la trazabilidad de los requisitos.
- Cada Sprint debe producir un incremento potencialmente desplegable, acompañado de pruebas, revisión técnica y verificación de seguridad.

11.2 Roles Scrum y Responsabilidades en MediSinc-IA

Tabla 14. Roles Scrum y responsabilidades específicas del proyecto.

Rol	Responsabilidades principales	Aplicación en MediSinc-IA
Product Owner (PO)	Representar a los interesados; maximizar el valor del producto; priorizar y ordenar el Product Backlog; validar los incrementos.	Priorizar funcionalidades clínicas y de seguridad por encima de mejoras puramente estéticas; validar el dashboard, el flujo de pre-triage y los criterios de aceptación.
Scrum Master	Facilitar las ceremonias; eliminar impedimentos; promover las prácticas de Scrum; proteger el foco del equipo.	Resolver bloqueos relacionados con Supabase, APIs de IA, despliegue, integración frontend/backend y coordinación del trabajo.

Rol	Responsabilidades principales	Aplicación en MediSinc-IA
Equipo de Desarrollo	Autoorganizarse; diseñar, programar, probar, integrar y documentar los incrementos.	Daniel Frias Vargas, Jose Maria Peredo Barba y Emiliano Arnechino Ardaya trabajan sobre React 18, FastAPI/Python, Supabase PostgreSQL, seguridad e integración de IA.

11.3 Product Backlog Detallado (Pila del Producto)

El Product Backlog es un artefacto vivo. Las historias se estiman mediante Puntos de Historia basados en la sucesión de Fibonacci, considerando complejidad, incertidumbre y esfuerzo. La gestión puede realizarse en un tablero colaborativo, manteniendo trazabilidad desde “To Do” hasta “Done”.

Tabla 15. *Product Backlog de MediSinc-IA.*

ID	Historia de Usuario (User Story)	Criterios de Aceptación	Prioridad / Puntos
US01	Como sistema, quiero cifrar el CI y generar un hash HMAC para asegurar la privacidad de datos.	- Implementación de security.py. - Pruebas unitarias de encriptación exitosas. - Hash no reversible.	Crítica / 5 pts
US02	Como administrador, quiero crear cuentas de doctores y auditar logs inmutables para garantizar gobernanza.	- Tabla PROFILES con RLS activa. - Endpoints protegidos por JWT. - Toda acción genera traza en AUDIT_LOG.	Crítica / 8 pts
US03	Como paciente, quiero un formulario para ingresar mis datos fijos y síntoma base sin registro previo.	- Formulario UI responsivo. - Validación de edad (0-120) y CI. - Payload JSON empaquetado correctamente.	Alta / 3 pts
US04	Como paciente, quiero responder preguntas adaptativas de IA basadas en mi síntoma principal.	- Invocación exitosa a ai_factory.py. - Renderizado de 2 a 3 preguntas dinámicas. - Fallback configurado.	Alta / 8 pts
US05	Como sistema, quiero estructurar respuestas del paciente usando IA agnóstica para homologar la información clínica.	- Uso estricto de Pydantic AIStructuredOutput. - Diccionario de modismos bolivianos integrado.	Alta / 5 pts
US06	Como sistema, quiero un motor de reglas (Safety Overrides) que fuerce ROJO ante síntomas críticos.	- Intercepción de IA exitosa. - Casos de lactante febril detectados. - override_applied registrado en BD.	Crítica / 8 pts

ID	Historia de Usuario (User Story)	Criterios de Aceptación	Prioridad / Puntos
US07	Como paciente, quiero recibir un código alfanumérico único y QR interactivo como comprobante.	- Generación de código formato MS-XXXXX. - Generación de código QR en canvas HTML. - Estado en BD pasa a RECEIVED.	Alta / 3 pts
US08	Como doctor, quiero ver un Dashboard priorizado (Rojo, Amarillo, Verde) para atender urgencias primero.	- Endpoint /dashboard funcional. - UI de tarjetas ordenadas por prioridad y timestamp.	Alta / 5 pts
US09	Como doctor, quiero escanear el QR del paciente con la cámara web para agilizar la búsqueda.	- Integración de html5-qrcode. - Búsqueda en < 1 seg mediante hash o MS-XXXXX.	Media / 3 pts
US10	Como doctor, quiero revisar la ficha clínica en pantalla dividida y registrar notas para cerrar atención.	- Modal UI Split View (Raw vs. IA). - Registro en MEDICAL_REVIEW. - Estado paciente actualizado a REVIEWED.	Alta / 8 pts

11.4 Criterios de Priorización y Preparación de Historias

Para entrar a un Sprint, una historia debe tener un objetivo comprensible, criterios de aceptación verificables, dependencias identificadas y una estimación acordada por el equipo. La prioridad se determina principalmente por impacto clínico, seguridad, dependencia técnica y valor para el usuario.

Tabla 16. *Criterios para ordenar y preparar el Product Backlog.*

Criterio	Regla de aplicación
Impacto clínico	Las funciones que reducen el riesgo de no detectar una emergencia tienen prioridad superior.
Seguridad y privacidad	Cifrado, RLS, autenticación, auditoría y protección de PII se consideran trabajo crítico.
Dependencias técnicas	Las historias necesarias para habilitar otras funcionalidades deben abordarse primero.
Valor para el usuario	Se priorizan flujos que reduzcan tiempos de admisión y mejoren la atención médica.
Verificabilidad	Cada historia debe poder validarse mediante criterios de aceptación y pruebas.

11.5 Sprint Planning y Ejecución (Iteraciones)

Se proponen tres Sprints de desarrollo, con una duración recomendada de dos días cada uno. La distribución siguiente es una propuesta editable basada en las dependencias funcionales y las prioridades declaradas en el Product Backlog.

Tabla 17. Plan de Sprints propuesto para MediSinc-IA.

Sprint	Duración	Objetivo	Historias principales	Incremento esperado
Sprint 1	2 días	Construir la base segura del sistema y el flujo inicial del paciente.	US01, US02, US03, US04	Registro de paciente, seguridad de CI, cuentas médicas, preguntas adaptativas y base de integración con IA.
Sprint 2	2 días	Consolidar la estructuración clínica y la seguridad determinista.	US05, US06, US07	Salida clínica estructurada, Safety Overrides, generación de código MS-XXXXX y QR.
Sprint 3	2 días	Completar el flujo operativo del médico y el cierre de atención.	US08, US09, US10	Dashboard priorizado, búsqueda/escaneo QR, revisión de ficha, notas médicas y estado REVIEWED.

11.6 Roadmap editable de seis días

Tabla 18. Roadmap de ejecución en seis días.

Actividad / Semana	D1	D2	D3	D4	D5	D6
Sprint 1 – Fundamentos y seguridad	•	•				
Sprint 2 – IA y Safety Overrides			•	•		
Sprint 3 – Operación médica					•	•
Pruebas e integración	•	•	•	•	•	•

Actividad / Semana	D1	D2	D3	D4	D5	D6
Documentación y revisión	•	•	•	•	•	•

11.7 Ceremonias y Dinámicas de Equipo

Tabla 19. Ceremonias Scrum y resultados esperados.

Ceremonia	Momento	Propósito	Resultado
Sprint Planning	Inicio de cada Sprint	Evaluar capacidad, seleccionar historias y definir el Sprint Goal.	Sprint Backlog y objetivo del Sprint.
Daily Scrum	Diaria, máximo 15 minutos	Inspeccionar el progreso y detectar impedimentos o dependencias.	Ajustes de coordinación y acciones de desbloqueo.
Sprint Review	Final de cada Sprint	Demostrar software funcional y recoger retroalimentación del Product Owner.	Feedback y actualización del Product Backlog.
Sprint Retrospective	Final de cada Sprint	Identificar mejoras en herramientas, procesos y coordinación.	Acciones de mejora para el siguiente Sprint.

11.8 Definition of Done (DoD - Definición de Terminado)

Tabla 20. Definition of Done del proyecto MediSinc-IA.

Criterio	Condición para considerar terminada una historia
Revisión de código	El código ha pasado por revisión de pares (Code Review).
Seguridad de datos	Las políticas RLS en Supabase están habilitadas y verificadas para las nuevas tablas.
Protección de PII	No existen exposiciones en texto plano de información personal identificable como el Carnet de Identidad.
Pruebas	Los criterios de aceptación y las pruebas correspondientes se ejecutan satisfactoriamente.
Integración	El incremento está integrado con las ramas y componentes correspondientes sin errores de compilación.
Despliegue	El frontend puede desplegarse en staging/producción sin errores de compilación de Vite.
Documentación	El cambio relevante está documentado y mantiene coherencia con la arquitectura y los requisitos.
Repositorio	El código se encuentra actualizado en el repositorio principal (Git).

Criterio	Condición para considerar terminada una historia
UX responsiva	La interfaz mantiene consistencia visual y comportamiento responsivo en dispositivos móviles y de escritorio.

11.9 Seguimiento y Control del Sprint

El avance debe revisarse mediante el estado de las historias, los impedimentos registrados, las pruebas ejecutadas y el incremento funcional disponible. Las métricas pueden utilizarse como apoyo para inspección y adaptación, sin convertirlas en objetivos aislados del valor clínico.

Tabla 21. *Indicadores de seguimiento y control.*

Indicador	Uso
Historias terminadas	Medir el cumplimiento del Sprint Goal.
Puntos completados	Observar la capacidad histórica del equipo y mejorar la estimación.
Defectos abiertos	Controlar la calidad antes de considerar un incremento terminado.
Pruebas PASS/FAIL	Verificar objetivamente los criterios de aceptación.
Impedimentos	Registrar bloqueos y asegurar su resolución o escalamiento.

Referencias

- American Psychological Association. (2020). Publication manual of the American Psychological Association (7th ed.). American Psychological Association.
- Institute of Electrical and Electronics Engineers. (1998). IEEE recommended practice for software requirements specifications (IEEE Std 830-1998). IEEE.
- Schwaber, K., & Sutherland, J. (2020). The Scrum Guide: The definitive guide to Scrum: The rules of the game. Scrum.org.